



SDEV2171

Lesson 12: Fetching Remote Data

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain what an API request does in a mobile app
- Explain when a request should run in a screen
- Use one explicit trigger to fetch remote JSON data
- Store fetched data in state and update the UI
- Debug common request and asynchronous issues

Lesson Roadmap

1. Reconnect lesson-11 debugging habits to API work
2. Define APIs, endpoints, JSON, and async responses
3. Choose one safe request trigger
4. Guided build of one remote-data screen
5. Testing and troubleshooting
6. Checkpoint and lesson-13 handoff

Starting Point From Lessons 10 And 11

- use the same Expo Router project shape from lessons 10 and 11
- confirm the home screen still opens the detail screen
- confirm the detail screen and back navigation still work before you add network code

If your project is not ready

- if your baseline is `resolved` , keep using your own project
- if your baseline is `partial` or `blocked` , recover the working list baseline first
- if recovery will take too long, switch to the instructor-guided reference path and record your next step

Why Remote Data Matters

- many apps do not store all useful data directly in the screen
- weather, maps, schedules, and social feeds all ask another system for data
- today the screen still owns local state
- the difference is that the state will be filled by a network request

Core Terms

Term	Plain-language meaning
API	a service that gives data to apps
endpoint	one URL you request from that service
request	asking the endpoint for data
response	what comes back
JSON	common text format used for API data

What The Request Is Doing

- the app sends one request to ask another system for data
- the API sends back a response with JSON content
- the screen does not render the remote rows until that response has been turned into screen state

Quick class prompt

- What is this request asking for?
- Why does this app need an API instead of only local hard-coded data?

Async Means Not Immediate

- a request does not finish instantly
- the screen asks for data, then waits for the response
- your code must be ready for “not here yet” values

Important boundary

- lesson 12 builds the request and success path
- lesson 13 adds full loading and error UI

The Trigger Rule

- do not start a request every time the component renders
- choose one clear reason for the request to run

Today's rule

- fetch once when the detail screen loads
- use one `useEffect` with one stable dependency rule

Wrong Trigger Vs Right Trigger

Wrong for this lesson

- start `fetch()` directly in the render body
- omit the dependency array and let the Effect run after every render

Right for this lesson

- keep the request helper separate
- trigger it once from one `useEffect` with `[]`

Why Repeated Requests Happen

- if a request starts directly in the render body, it can run again after each state update
- if an Effect has the wrong dependency pattern, it can also re-run too often

Simple warning sign

- the terminal or network logs keep repeating
- the screen never settles into one stable list

The Data Flow Today

1. the screen loads
2. `useEffect` runs one request helper
3. `fetch()` asks the endpoint for JSON
4. the response is checked and parsed
5. the result is transformed into list items
6. `setState` updates the list UI

Minimal Fetch Pattern

```
const response = await fetch('https://jsonplaceholder.typicode.com/todos?_limit=6');

if (!response.ok) {
  throw new Error(`Response status: ${response.status}`);
}

const data = await response.json();
```

What to notice

- `fetch` returns a promise
- `response.ok` still needs to be checked
- `response.json()` is also asynchronous

Main Lesson Trigger Pattern

```
useEffect(() => {  
  loadRemoteSteps();  
}, []);
```

What this means

- the request is tied to screen load
- the empty dependency array prevents a run-after-every-render loop
- the request logic stays out of the render body

One Safe Helper Shape

```
async function loadRemoteSteps() {  
  const response = await fetch('https://jsonplaceholder.typicode.com/todos?_limit=6');  
  
  if (!response.ok) {  
    throw new Error(`Response status: ${response.status}`);  
  }  
  
  const data = await response.json();  
  
  setRemoteSteps(  
    data.map((item) => ({  
      id: String(item.id),  
      label: item.title,  
    })))  
  );  
}
```



ESSENTIAL
TO ALBERTA



Quick Sort Before You Edit

State you still need

- the array that will hold fetched list items

Logic you need to add

- one request helper
- one trigger for when that helper runs

UI you can keep

- the list rendering pattern
- the back-navigation path

Guided Build Order

1. confirm the working list-screen baseline
2. open `src/app/details.js`
3. keep the working local list visible until the request helper is proven
4. add one async request helper
5. log one response shape
6. add one screen-load `useEffect`
7. confirm the request runs once
8. map the response into the same list-row shape
9. replace the old local array with remote state once the response path is working
10. confirm the list renders returned data

What To Remove And What To Keep

Remove

- the old local starter array
- add-step and clear-list controls from lesson 10

Keep

- the detail screen route
- the list rendering pattern
- the back button
- your debugging habits from lesson 11



ESSENTIAL
TO ALBERTA



One Full Example Loop

Symptom

- the screen opens, but no remote list items appear

First checks

- did the request helper run at all?
- what does the logged response data look like?
- does the mapped object use the same `label` key the list expects?

Small fix pattern

- correct one request, mapping, or trigger issue
- retest the same screen load

Common Request Risks Today

- request started in the render body
- missing empty dependency array
- response shape does not match the list-row shape
- `response.ok` never checked
- state updated with the wrong property name

Tool Clues To Use

- one `console.log` of the parsed JSON or one item
- one terminal error line if the request fails
- one DevTools Console observation from the development-build path if needed

Rule

- do not guess whether the request worked
- inspect one real clue first

Mid-Build Checkpoint

Students should be ready to show

- one clear endpoint URL
- one named trigger rule for when the request runs
- one short explanation of what the request is asking for
- one logged response clue
- one state update that turns response data into list data

Students should also be able to answer

- Why does this request run on screen load?
- Why does this app need an API here?
- What would cause it to repeat too often?
- What shape does one rendered list item need?

What We Learned Today

- remote data arrives through API requests
- `fetch` plus `response.ok` plus `json()` is the basic success path
- request timing matters as much as request syntax
- `useEffect` is one way to start screen-load work at the right time
- response data still has to match the UI shape your list expects

Lesson 13 Bridge

- next lesson improves the user experience around this request
- new states will be added:
 - loading
 - success
 - error

Keep from today

- one clear trigger
- one clear response shape
- one clear debugging loop

Next Step

- keep the request working in a known-good state
- keep logs only if they still help
- be ready to add logging and error feedback next lesson